AI Assisted Coding
Lab Assignment - 12.5

Name : Shivanand Rama
Hall.t.No. : 2303A51037
Batch No. : 01

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms
Lab Objectives:

- Apply AI-assisted programming to implement and optimize sorting and searching algorithms.
- Compare different algorithms in terms of efficiency and use cases.
- Understand how AI tools can suggest optimized code and complexity improvements.

Task Description #1 (Sorting – Merge Sort Implementation)

- **Task: Use AI to generate a Python program that implements the Merge Sort algorithm.**
- **Instructions:**
  - o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
  - o Ask AI to include time complexity and space complexity in the function docstring.
  - o Verify the generated code with test cases.
- **Expected Output:**
  - o A functional Python script implementing Merge Sort with proper documentation.

Copilot Suggested Code :

task_1.py > ...

```python
# Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

from pandas import merge


def merge_sort(arr):
    """
    Sorts a list in ascending order using the Merge Sort algorithm.

    Time Complexity: O(n log n) - where n is the number of elements in the array.
    Space Complexity: O(n) - due to the temporary arrays used for merging.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new list containing the sorted elements.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)

def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.

    Parameters:
    left (list): The first sorted list.
    right (list): The second sorted list.

    Returns:
    list: A new list containing the merged and sorted elements from both lists.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    # If there are remaining elements in left or right, add them to merged
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged

# Test cases to verify the implementation
if __name__ == "__main__":
    test_cases = [
        [38, 27, 43, 3, 9, 82, 10],
        [5, 2, 9, 1, 5, 6],
        [12, 11, 13, 5, 6, 7],
        [],
        [1],
        [2, 1]
    ]

    for i, test in enumerate(test_cases):
        sorted_arr = merge_sort(test)
        print(f"Test Case {i + 1}: {sorted_arr}")
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-12.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-12.5/task_1.py
Test Case 1: [3, 9, 10, 27, 38, 43, 82]
Test Case 2: [1, 2, 5, 5, 6, 9]
Test Case 3: [5, 6, 7, 11, 12, 13]
Test Case 4: []
Test Case 5: [1]
Test Case 6: [1, 2]
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- **Task: Use AI to create a binary search function that finds a target element in a sorted list.**
- **Instructions:**
  - o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.
  - o Include docstrings explaining best, average, and worst-case complexities.
  - o Test with various inputs.
- **Expected Output:**
  - o Python code implementing binary search with AI- generated comments and docstrings.

Copilot Suggested Code :

task_2.py > ...

```python
# Task: Use AI to create a binary search function that finds a target element in a sorted list.

def binary_search(arr, target):
    """
    Performs binary search on a sorted list to find the index of the target element.

    Time Complexity:
    - Best Case: O(1) - when the target is at the middle of the array.
    - Average Case: O(log n) - where n is the number of elements in the array.
    - Worst Case: O(log n) - when the target is not present in the array.

    Space Complexity: O(1) - as we are using a constant amount of space.

    Parameters:
    arr (list): A sorted list of elements.
    target: The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    # Target was not found in the array
    return -1

# Test cases to verify the implementation
if __name__ == "__main__":
    test_cases = [
        ([1, 2, 3, 4, 5], 3),  # Target is present
        ([1, 2, 3, 4, 5], 6),  # Target is not present
        ([1, 2, 3, 4, 5], 1),  # Target is the first element
        ([1, 2, 3, 4, 5], 5),  # Target is the last element
        ([], 1),               # Empty array
    ]

    for arr, target in test_cases:
        result = binary_search(arr, target)
        print(f"Searching for {target} in {arr} -> Index: {result}")
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-12.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-12.5/task_2.py
Searching for 3 in [1, 2, 3, 4, 5] -> Index: 2
Searching for 6 in [1, 2, 3, 4, 5] -> Index: -1
Searching for 1 in [1, 2, 3, 4, 5] -> Index: 0
Searching for 5 in [1, 2, 3, 4, 5] -> Index: 4
Searching for 1 in [] -> Index: -1
```

Task Description #3: Smart Healthcare Appointment Scheduling System
A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. **Search appointments using appointment ID.**
2. **Sort appointments based on time or consultation fee.**

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

Copilot Suggested Code :

```python
# Task Description #3: Smart Healthcare Appointment Scheduling System A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system need
# 1. Search appointments using appointment ID.
# 2. Sort appointments based on time or consultation fee.

# Recommended Algorithms:
# 1. Searching Algorithm: Hash Table (Dictionary in Python)
# 2. Sorting Algorithm: Merge Sort
# Justification:
# 1. Hash Table (Dictionary): Provides O(1) average-case time complexity for searching, making it highly efficient for finding appointments by ID.
# 2. Merge Sort: Offers O(n log n) time complexity for sorting, ensuring efficient ordering of appointments based on time or consultation fee.
# Implementation:
class Appointment:
    def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
        self.appointment_id = appointment_id
        self.patient_name = patient_name
        self.doctor_name = doctor_name
        self.appointment_time = appointment_time
        self.consultation_fee = consultation_fee

class HealthcarePlatform:
    def __init__(self):
        self.appointments = {}

    def add_appointment(self, appointment):
        self.appointments[appointment.appointment_id] = appointment

    def search_appointment(self, appointment_id):
        return self.appointments.get(appointment_id, None)

    def sort_appointments_by_time(self):
        return sorted(self.appointments.values(), key=lambda x: x.appointment_time)

    def sort_appointments_by_fee(self):
        return sorted(self.appointments.values(), key=lambda x: x.consultation_fee)

# Test cases to verify the implementation
if __name__ == "__main__":
    platform = HealthcarePlatform()

    # Adding sample appointments
    platform.add_appointment(Appointment(1, "Alice", "Dr. Smith", "2024-07-01 10:00", 100))
    platform.add_appointment(Appointment(2, "Bob", "Dr. Jones", "2024-07-01 11:00", 150))
    platform.add_appointment(Appointment(3, "Charlie", "Dr. Brown", "2024-07-01 09:00", 120))

    # Searching for an appointment
    appointment = platform.search_appointment(2)
    if appointment:
        print(f"Found appointment: {appointment.patient_name} with {appointment.doctor_name} at {appointment.appointment_time} for ${appointment.consultation_fee}")
    else:
        print("Appointment not found.")

    # Sorting appointments by time
    sorted_by_time = platform.sort_appointments_by_time()
    print("Appointments sorted by time:")
    for appt in sorted_by_time:
        print(f"{appt.patient_name} with {appt.doctor_name} at {appt.appointment_time} for ${appt.consultation_fee}")

    # Sorting appointments by consultation fee
    sorted_by_fee = platform.sort_appointments_by_fee()
    print("Appointments sorted by consultation fee:")
    for appt in sorted_by_fee:
        print(f"{appt.patient_name} with {appt.doctor_name} at {appt.appointment_time} for ${appt.consultation_fee}")
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-12.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-12.5/task_3.py
Found appointment: Bob with Dr. Jones at 2024-07-01 11:00 for $150
Appointments sorted by time:
Charlie with Dr. Brown at 2024-07-01 09:00 for $120
Alice with Dr. Smith at 2024-07-01 10:00 for $100
Bob with Dr. Jones at 2024-07-01 11:00 for $150
Appointments sorted by consultation fee:
Alice with Dr. Smith at 2024-07-01 10:00 for $100
Charlie with Dr. Brown at 2024-07-01 09:00 for $120
Bob with Dr. Jones at 2024-07-01 11:00 for $150
```

Task Description #4: Railway Ticket Reservation System Scenario A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. **Search tickets using ticket ID.**
2. **Sort bookings based on travel date or seat number.**

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

Copilot Suggested Code :

```python
# Task Description #4: Railway Ticket Reservation System Scenario A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:
# 1. Search tickets using ticket ID.
# 2. Sort bookings based on travel date or seat number.

# Recommended Algorithms:
# 1. Searching Algorithm: Hash Table (Dictionary in Python)
# 2. Sorting Algorithm: Merge Sort
# Justification:
# 1. Hash Table (Dictionary): Provides O(1) average-case time complexity for searching, making it highly efficient for finding tickets by ID.
# 2. Merge Sort: Offers O(n log n) time complexity for sorting, ensuring efficient ordering of bookings based on travel date or seat number.
# Implementation:
class Ticket:
    def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
        self.ticket_id = ticket_id
        self.passenger_name = passenger_name
        self.train_number = train_number
        self.seat_number = seat_number
        self.travel_date = travel_date

class RailwayReservationSystem:
    def __init__(self):
        self.tickets = {}

    def add_ticket(self, ticket):
        self.tickets[ticket.ticket_id] = ticket

    def search_ticket(self, ticket_id):
        return self.tickets.get(ticket_id, None)

    def sort_tickets_by_date(self):
        return sorted(self.tickets.values(), key=lambda x: x.travel_date)

    def sort_tickets_by_seat_number(self):
        return sorted(self.tickets.values(), key=lambda x: x.seat_number)

# Test cases to verify the implementation
if __name__ == "__main__":
    system = RailwayReservationSystem()

    # Adding sample tickets
    system.add_ticket(Ticket(1, "Alice", "Train A", "12A", "2024-08-01"))
    system.add_ticket(Ticket(2, "Bob", "Train B", "10B", "2024-08-02"))
    system.add_ticket(Ticket(3, "Charlie", "Train A", "11C", "2024-08-01"))

    # Searching for a ticket
    ticket = system.search_ticket(2)
    if ticket:
        print(f"Found ticket: {ticket.passenger_name} on {ticket.train_number} seat {ticket.seat_number} for travel on {ticket.travel_date}")
    else:
        print("Ticket not found.")

    # Sorting tickets by travel date
    sorted_by_date = system.sort_tickets_by_date()
    print("Tickets sorted by travel date:")
    for t in sorted_by_date:
        print(f"{t.passenger_name} on {t.train_number} seat {t.seat_number} for travel on {t.travel_date}")

    # Sorting tickets by seat number
    sorted_by_seat = system.sort_tickets_by_seat_number()
    print("Tickets sorted by seat number:")
    for t in sorted_by_seat:
        print(f"{t.passenger_name} on {t.train_number} seat {t.seat_number} for travel on {t.travel_date}")
```

Output :

```
PS C:\B.Tech\3-2\AI_Assisted_Coding\lab_assignment-12.5> & "C:\Program Files\Python314\python.exe" c:/B.Tech/3-2/AI_Assisted_Coding/lab_assignment-12.5/task_4.py
Found ticket: Bob on Train B seat 10B for travel on 2024-08-02
Tickets sorted by travel date:
Alice on Train A seat 12A for travel on 2024-08-01
Charlie on Train A seat 11C for travel on 2024-08-01
Bob on Train B seat 10B for travel on 2024-08-02
Tickets sorted by seat number:
Bob on Train B seat 10B for travel on 2024-08-02
Charlie on Train A seat 11C for travel on 2024-08-01
Alice on Train A seat 12A for travel on 2024-08-01
```